PaperPilot – GenAI Research Paper Assistant

PROJECT PROGRESS REPORT

Team: Team-1

Team Members: Harish Kumar (Lead Developer - AI & Backend)
Kalpana Bhardwaj (Frontend & UI/UX)
Vansh (Research & Requirement Analysis)

Faculty Superviser: Mr. Saqib

Mentor: Mr. Zuber

University: Jagannath University

Project Live Tracking & Source Code:

- **GitHub Repository: https://github.com/Harishyadav44/PaperPilot-GenAI**
- **Jira Scrum Board: PG Sprint 1 - PaperPilot GenAI - Scrum Board - Jira**

1. Project Abstract

PaperPilot is an advanced, web-based GenAI Research Assistant designed to streamline the academic research process. By leveraging Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs), the system allows users to upload lengthy research papers and instantly extract context-aware insights, summaries, and structured notes.

2. Technology Stack Implemented

- **Frontend Interface:** Streamlit (Python)
- **Core Backend Pipeline:** LangChain Framework
- **LLM Engine:** Groq API (Model: llama-3.3-70b-versatile)
- **Vector Database & Embeddings:** ChromaDB and HuggingFace (all-MiniLM-L6-v2)
- **PDF Processing:** PyPDF2

3. System Architecture Design

The core of PaperPilot operates on a strict RAG workflow to ensure the AI generates answers based solely on the uploaded document context.

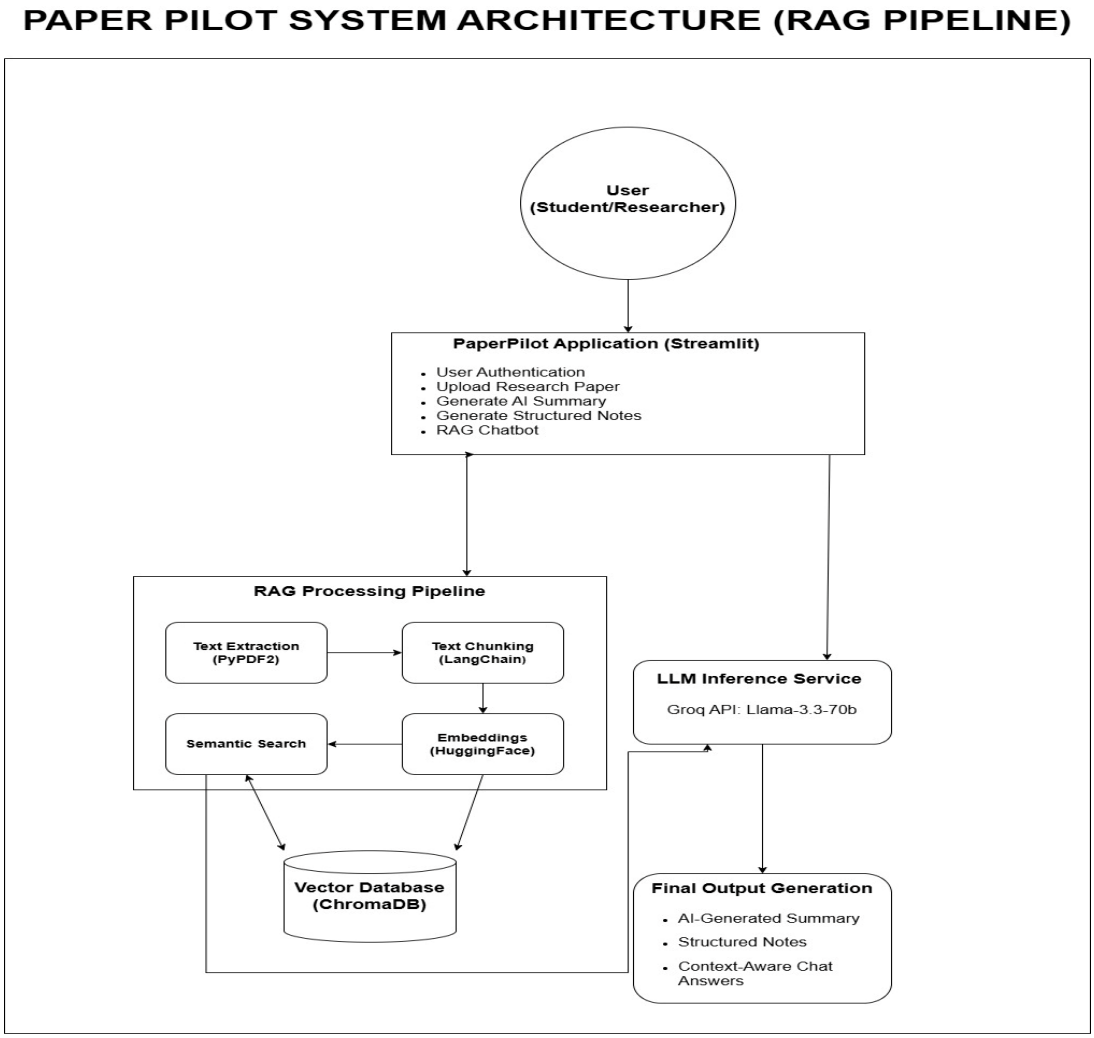

Figure 1: High-Level System Architecture and RAG (Retrieval-Augmented Generation) Pipeline of PaperPilot.

Workflow Breakdown

- **Ingestion:**
  Text is extracted from the uploaded PDF document.
- **Processing:**
  Text is chunked and converted into vector embeddings.
- **Storage:**
  Embeddings are securely stored in a local ChromaDB instance.
- **Retrieval & Generation:**
  User queries trigger a semantic search, and the relevant context is passed to the Groq API (Llama-3) to generate a highly accurate, natural language response.

4. 6-Week Agile Development Plan (Progress Track)

The project execution strictly follows the proposed 6-week development plan. Below is the detailed status report up to the current review phase (End of Week 4).

Phase 1: Week 1 – Planning (Completed)

- **Sprint Goal:**
  Define project scope and architecture.
- **Key Tasks Executed:**
  Requirement gathering, defining PaperPilot features, selecting the technology stack, creating the GitHub repository, and designing the architecture diagram.
- **Deliverables Achieved:**
  Software Requirements Specification (SRS) document, finalized Architecture Diagram, and initial Product Backlog.

Phase 2: Week 2 – Prototype (Completed)

- **Sprint Goal:**
  Build a basic working prototype.
- **Key Tasks Executed:**
  Designed UI wireframes, completed initial backend setup, developed the PDF upload module, and integrated basic LLM summarization capabilities.
- **Deliverables Achieved:**
  A working Python prototype capable of summarizing research papers.

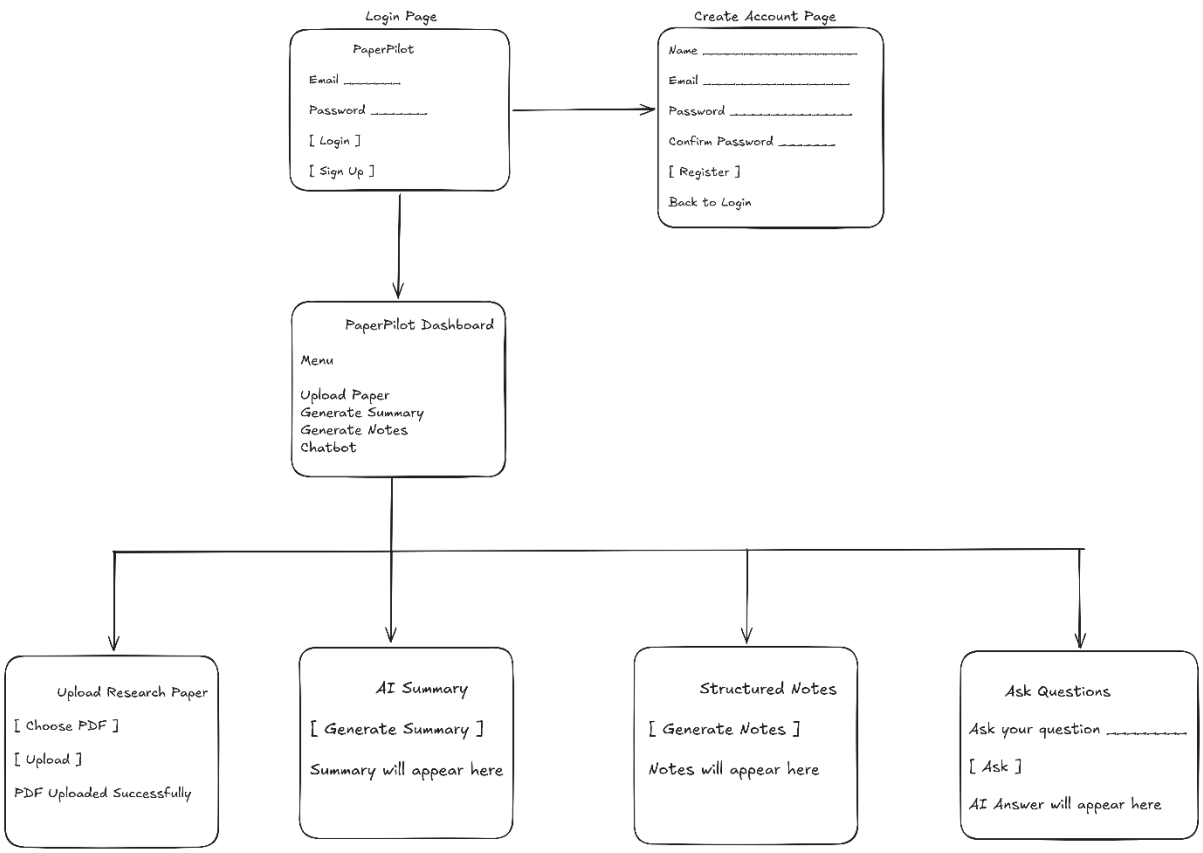

Figure 2: Low-Fidelity UI Wireframes and Dashboard Navigation Flow.

Phase 3: Week 3 – AI Features (Completed)

- **Sprint Goal:**
  Develop the core GenAI pipeline.
- **Key Tasks Executed:**
  Successfully implemented the RAG pipeline, created embeddings, set up the vector database, and developed the core logic for the Q&A chatbot.
- **Deliverables Achieved:**
  A functional backend paper Q&A chatbot.

Phase 4: Week 4 – Integration (Recently Completed – Current Review Phase)

- **Sprint Goal:**
  Integrate frontend and backend systems.
- **Key Tasks Executed:**
  Built the full Streamlit UI dashboard, connected all backend APIs to the frontend, successfully displayed AI summaries, and added the interactive chat interface.
- **Deliverables Achieved:**
  A fully integrated, functional web application system ready for review.

5. Current System Status (As of Week 4)

The PaperPilot application has successfully achieved its primary objective. The frontend UI and backend RAG pipeline are fully integrated. Users can securely upload a PDF, generate an AI summary, extract structured notes, and ask direct questions via the chatbot interface.

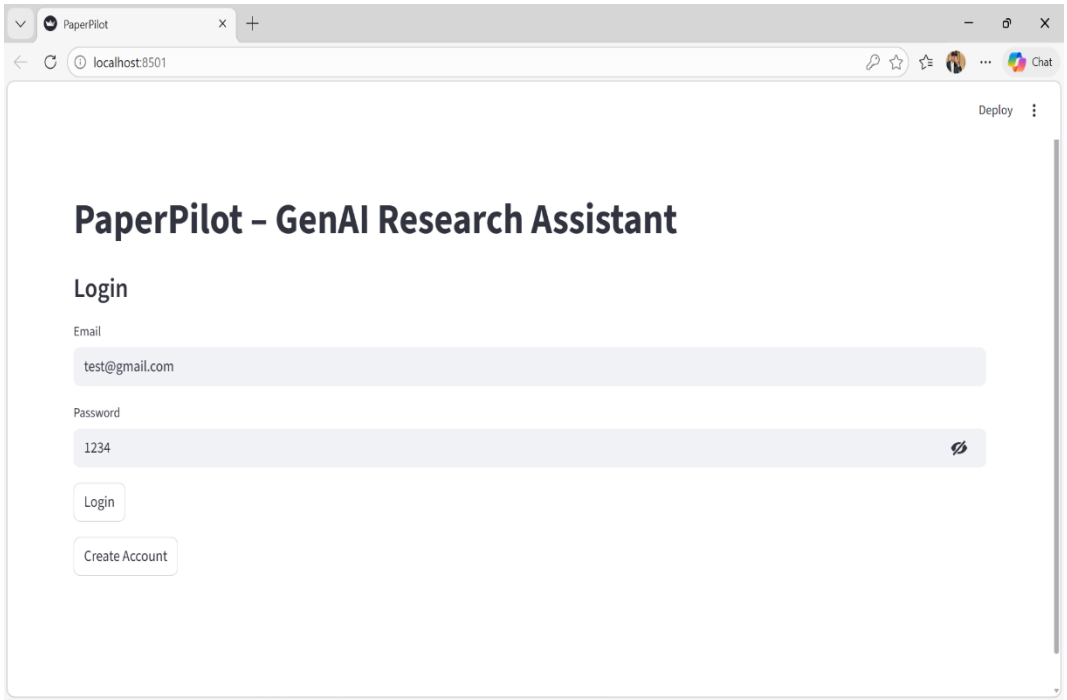

Figure 3: Secure User Authentication and Login Interface.

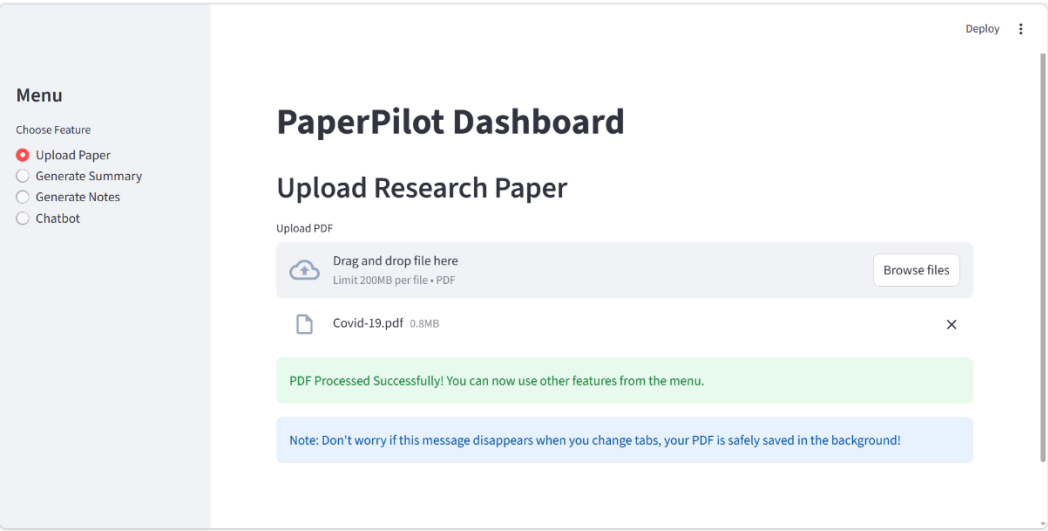

Figure 4: Document Ingestion and Automated PDF Processing Module.

Figure 5: Automated Context-Aware Summarization using Groq Llama-3.

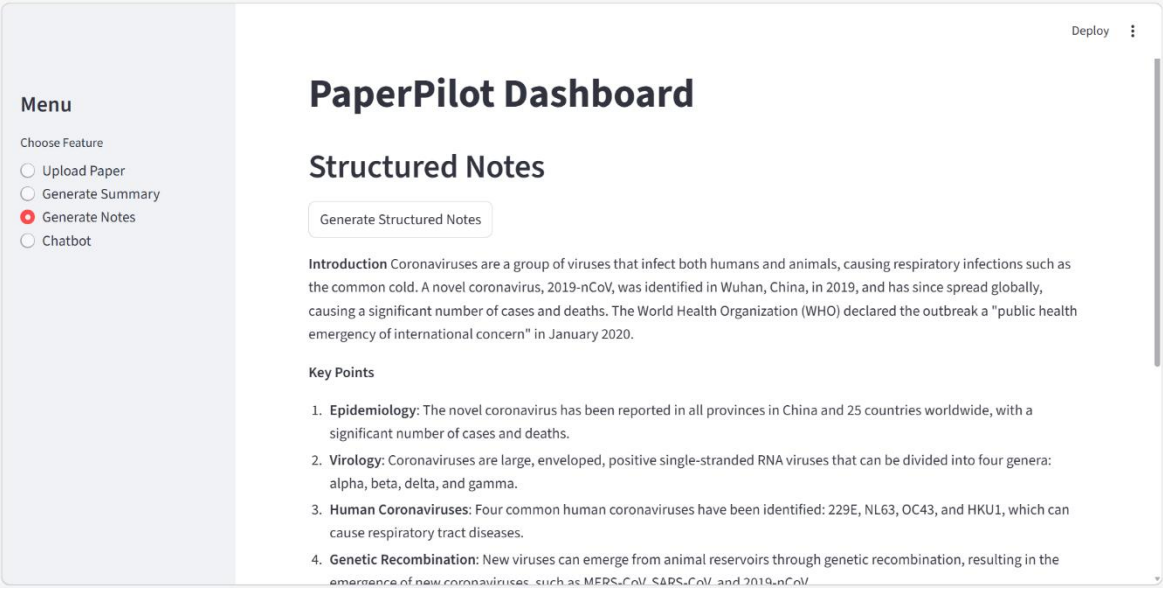

Figure 6: Extraction of Key Points and Structured Notes from the uploaded document.

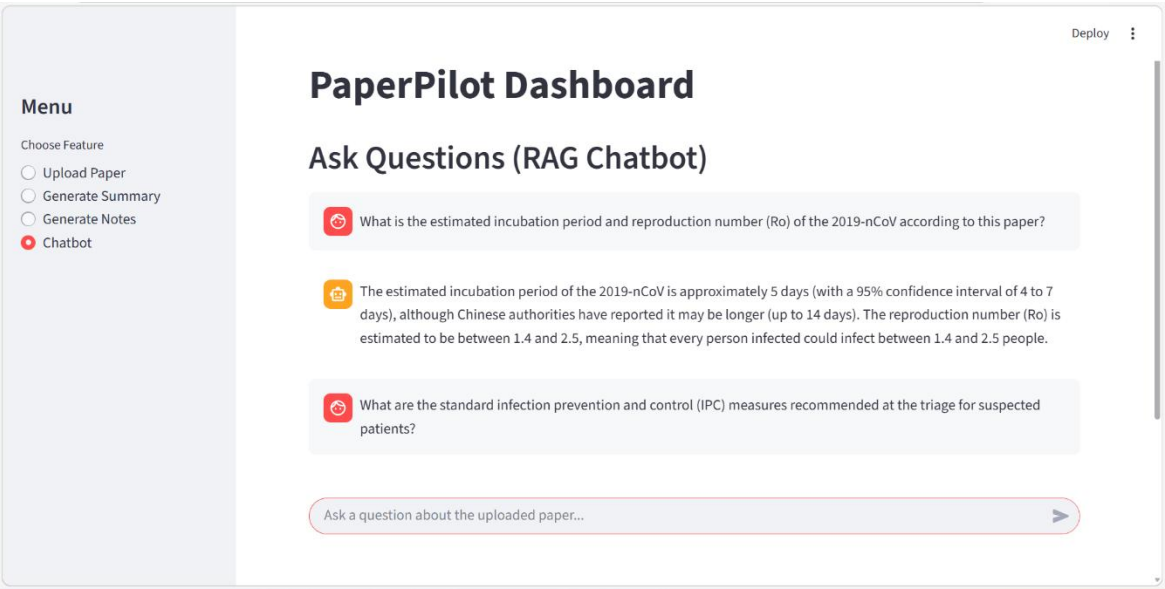

Figure 7.1

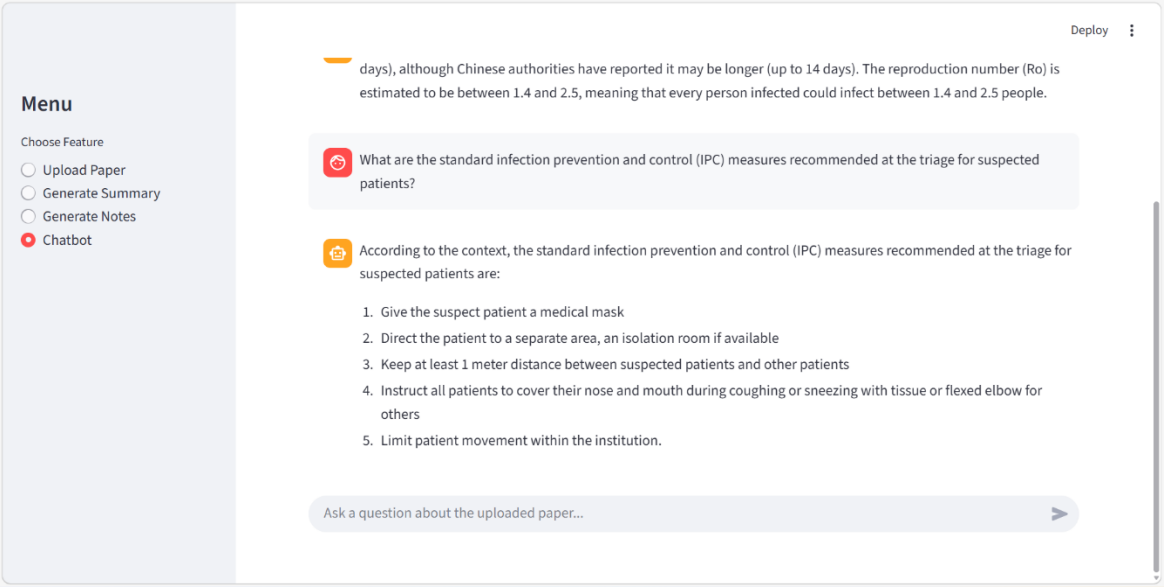

Figure 7.2

Figure 7(7.1, 7.2) : Real-time Interactive Q&A Chatbot retrieving exact technical data (Incubation Period & IPC Measures) from the Vector Database.

6. Upcoming Milestones (Pending Approval)

Upon successful review of the integrated system, the team will initiate the final two phases of the development plan:

- **Week 5 – Optimization:**
  Focus on bug fixing, improving LLM prompts, adding citation extraction, adding export functionality, and comprehensive system testing.
- **Week 6 – Deployment:**
  Finalizing the project report, deploying the system, and preparing the final presentation.